UCS CRM - Backend Security Review

Backend & API Security Review

Scope: The Express API in `backend/src` — middleware, routing, auth, headers, rate limiting, uploads, token handling and error handling.

Date: 2026-10-09 **Method:** Read the source and run `backend/scripts/security-check.mjs`. Every finding cites the file and line that proves it.

Related: database-layer issues (hardcoded DB credentials, unauthenticated `/api/db/*` routes, TLS, RDS exposure) are in DATABASE-SECURITY.md. This document covers the API itself.

---

1. Summary

ID	Severity	Finding	Status
BE-001	**Critical**	Developer-ticket routes accept any authenticated role	Open
BE-002	**High**	No security response headers (no `helmet`, no CSP/HSTS)	Open
BE-003	**High**	Wildcard CORS with an admin header allowed	Open
BE-004	**High**	No rate limiting on authentication endpoints	Open
BE-005	**High**	`/uploads` served statically with no authentication	Open
BE-006	**High**	`requireAdminKey` / env-admin fail **open** when the key is unset	Open
BE-007	**High**	`/api/customer/*` provisioning is unauthenticated	Open
BE-008	**Medium**	JWT verified without an algorithm allow-list (9 sites)	Open
BE-009	**Medium**	Long-lived tokens and fallback secrets	Open
BE-010	**Medium**	Server error details reach the client	Open
BE-011	**Medium**	`/api/fro-count` leaks worker PII unauthenticated	Open
BE-012	**Medium**	50 MB in-memory uploads (DoS)	Open
BE-013	Low	`trust proxy 'loopback'` does not match the real chain	Open

ID	Severity	Finding	Status
BE-014	Low	Default password `123456` ; admin password compared in plaintext	Open
BE-015	Low	No server-side schema validation	Open
BE-016	Info	Razorpay webhook has no replay/timestamp window	Open

Fix order: **BE-001** → **BE-006** → **BE-007** → **BE-002** → **BE-003** → **BE-004** → **BE-005**.

---

2. Findings

BE-001 — Developer-ticket routes accept any authenticated role

Severity: Critical · Confirmed

```
// backend/src/routes/developerTicketRoutes.js:11
router.use(authenticate);   // any valid JWT, any role
```

The router then exposes `GET /` (all tickets), `POST /`, `PUT /:id`, `PUT /bulk`, `PUT /:id/approve`, `PUT /:id/reject`, `PUT /:id/resolve`. A low-privilege account (e.g. a telecaller or worker token) can read and approve developer tickets. The client-side “ticket gate” is cosmetic: the shared credential lives in the bundle, and the server never checks it.

Remediation

```
import { authenticateRole } from '../middleware/authMiddleware.js';
router.use(authenticateRole('super_admin', 'developer'));
```

Rotate the shared credential in the client bundle; if a step-up check is truly wanted, implement it server-side as a short-lived challenge (never in JS).

BE-002 — No security response headers

Severity: High · Confirmed

There is no `helmet()` and no global header middleware. Searching the backend for `helmet`, `Content-Security-Policy`, `X-Frame-Options`, `Strict-Transport-Security` returns only one match — a `Content-Security-Policy` set on a single media route (`backend/src/index.js:896`). There is no

HSTS, no `X-Content-Type-Options`, and no framing protection, so the app can be framed by any origin (clickjacking) and XSS (if present) has no CSP backstop.

Remediation

```
import helmet from 'helmet';
app.use(helmet());
app.use(helmet.hsts({ maxAge: 15552000, includeSubDomains: true }));
// add a Content-Security-Policy appropriate to the panel bundles
```

BE-003 — Wildcard CORS with an admin header allowed

Severity: High · Confirmed

```
// backend/src/index.js:146-150
app.use(cors({
  origin: '*',
  methods: ['GET','POST','PUT','PATCH','DELETE','OPTIONS'],
  allowedHeaders: ['Content-Type','Authorization','X-Client-Type','x-admin-key'],
}));
```

Today auth is a bearer token in `localStorage`, so a cross-origin page cannot read it. The problem is the breadth and the direction of travel: the moment the token moves to a cookie (the correct fix for XSS-based theft), `origin: '*'` plus credentials becomes the exact CSRF-enabling combination. Allowing `x-admin-key` also invites a browser to send an admin key cross-origin.

Remediation: replace `'*'` with an explicit allow-list of the panel origins, drop `x-admin-key` from the browser-allowed headers, and never combine a wildcard origin with `credentials: true`.

BE-004 — No rate limiting on authentication endpoints

Severity: High · Confirmed

`express-rate-limit` is a dependency but is used in exactly one place: `backend/src/routes/chatRoutes.js:44`. There is no limiter on `/api/auth/login`, `/api/workers/login`, `/api/impersonation-codes`, the OTP endpoints, or `POST /api/whatsapp/send`. Password spraying and code guessing are unthrottled.

Remediation

```
import rateLimit from 'express-rate-limit';
const authLimiter = rateLimit({ windowMs: 15 * 60 * 1000, max: 5, standardHeaders: true
        });
app.use('/api/auth/login', authLimiter);
app.use('/api/workers/login', authLimiter);
```

Also add a coarse global limiter and set `trust proxy` correctly (BE-013).

BE-005 — `/uploads` served statically with no authentication

Severity: High · Confirmed

```
// backend/src/index.js:539
app.use('/uploads', express.static(path.resolve(__dirname, '../uploads')));
```

Mounted with no middleware. Every file in `uploads/` is world-readable if the path is known; uploaded documents include donor paperwork, bank statements, certificates and payment screenshots.

Remediation: move uploads outside the web root and serve them through a controller that checks ownership, or store them in S3 and stream only via a presigned, authorising route.

BE-006 — `requireAdminKey` / env-admin fail **open** when the key is unset

Severity: High · Confirmed

```
// backend/src/middleware/requireAdminKey.js:11
export const requireAdminKey = (req, res, next) => {
  if (!ADMIN_KEY) return next();   // fail OPEN
  ...
};
```

```
// backend/src/routes/envAdminRoutes.js:44
const requireKey = (req, res, next) => {
  if (!ENV_ADMIN_KEY) return next();   // fail OPEN
  ...
};
```

If `ENV_ADMIN_KEY` is not configured, both guards pass every request. The env-admin surface can read and write every project's `.env` and restart services. A missing environment variable silently disables the guard instead of denying access.

Remediation: fail closed.

```
if (!ADMIN_KEY) return res.status(503).json({ message: 'Not configured' });
```

BE-007 — `/api/customer/*` provisioning is unauthenticated

Severity: High · Confirmed

```
// backend/src/index.js:913 and :924 — no middleware
app.post('/api/customer/provision', async (req, res) => { /* creates DB + S3 + IAM */ });
app.get('/api/customer/list', async (req, res) => { /* ... */ });
```

`provisionCustomer()` creates a dedicated Postgres database, an S3 bucket and an IAM user. Exposed with no auth, this is a resource-creation and enumeration primitive for anyone who can reach the API.

Remediation: gate both routes with `authenticateRole('super_admin')` (and an admin key), or remove them from the public surface.

BE-008 — JWT verified without an algorithm allow-list

Severity: Medium · Confirmed

Every verification call passes no `algorithms` option, so the token's own header selects the algorithm:

File	Line
`backend/src/middleware/authMiddleware.js`	61, 77, 111, 128
`backend/src/socket.js`	57
`backend/src/metropad/middleware/auth.js`	23
`backend/src/controllers/whatsappCrmAuthController.js`	188
`backend/src/routes/dashboardRoutes.js`	24
`backend/src/routes/callLogRoutes.js`	11

Remediation: pin the algorithm everywhere.

```
jwt.verify(token, process.env.JWT_SECRET, { algorithms: ['HS256'] });
```

BE-009 — Long-lived tokens and fallback secrets

Severity: Medium · Confirmed

- `backend/src/config/db.js:1300` — `process.env.JWT_SECRET || 'dev-secret'`. If the env var is missing, tokens are signed with a publicly known secret.
- `backend/src/config/db.js:1301` — the same token is issued with `expiresIn: '100y'`.
- `backend/src/controllers/whatsappWebhookController.js:6` — `process.env.WHATSAPP_VERIFY_TOKEN || 'ucscompany123'`, a hardcoded fallback webhook token.

Remediation: delete both fallbacks (fail fast at boot when the env var is unset), and bound every token lifetime. Prefer short-lived access tokens plus a refresh token for long-lived integrations.

BE-010 — Server error details reach the client

Severity: Medium · Confirmed

`backend/src/index.js:200` returns `error.message` from the health check, and many handlers respond with `res.status(500).json({ message: err.message })` (e.g. `index.js:222`, `:569`, `:606`). Driver/SQL messages leak schema and library internals to any caller.

Remediation: return a stable error code and a generic message; log the detail server-side. The global handler at `index.js:1133` already returns a generic message — route the others through it.

BE-011 — `/api/fro-count` leaks worker PII unauthenticated

Severity: Medium · Confirmed

```
// backend/src/index.js:216 — no middleware
app.get('/api/fro-count', async (req, res) => {
  const { data: rows } = await db.from('workers').select('id, name, login_id,
        is_active').eq('department', 'fro');
  res.json({ fro_count: count, workers: rows });
});
```

Returns employee names and login IDs to anyone who can reach the API.

Remediation: add `authenticateRole('super_admin', 'admin', 'hr')`, or remove the diagnostic endpoint.

BE-012 — 50 MB in-memory uploads (DoS)

Severity: Medium · Confirmed

```
// backend/src/index.js:322
const uploadApi = multer({ storage: multer.memoryStorage(), limits: { fileSize: 50 * 1024
        * 1024 } });
```

Files are buffered fully in memory and the JSON body parser accepts 10 MB (`index.js:151-154`). Several concurrent 50 MB uploads can exhaust the 2 GB host.

Remediation: stream uploads to disk/S3 with `multer.diskStorage` or the S3 multipart upload, lower the cap per route, and enforce it server-side.

BE-013 — `trust proxy 'loopback'` does not match the real chain

Severity: Low · Confirmed

```
// backend/src/index.js:144
app.set('trust proxy', 'loopback');
```

Correct only for a single loopback hop. Behind nginx on another interface, `req.ip` resolves to the proxy, so any IP-based limiter or audit log groups all users together. This becomes a correctness bug the moment BE-004 is fixed.

Remediation: set the hop count to match the real chain (e.g. `1`) and verify the observed client IP in staging before enabling enforcement.

BE-014 — Default password `123456`; admin password compared in plaintext

Severity: Low · Confirmed

- `backend/src/models/crmAgentModel.js:20` — `DEFAULT_AGENT_PASSWORD = '123456'`.
- `backend/src/controllers/workerController.js:46` — returns `'123456'` as the default worker password.
- `backend/src/controllers/authController.js:222-223` — admin login compares `process.env.ADMIN_PASSWORD` with the submitted value using `===`.

Remediation: remove the default, require a minimum length, force a change on first login, and store the admin credential as a hash.

BE-015 — No server-side schema validation

Severity: Low · Confirmed

There is no schema-validation library on the server (`zod` is used only in the client), so request-body validation is ad-hoc and uneven by construction. Every route that accepts a body should type-check and bound its fields, coerce and cap pagination, and reject unknown fields on high-value endpoints.

BE-016 — Razorpay webhook has no replay/timestamp window

Severity: Info · Confirmed

`backend/src/services/razorpayWebhook.js:13-20` verifies the HMAC with `crypto.timingSafeEqual` (good), but there is no timestamp/replay check, so a captured signed payload can be replayed.

Remediation: validate the delivery timestamp against a short window and store processed event IDs to reject duplicates.

---

3. What is already done well

- Most routers use an explicit role gate (`authenticateRole(...)`); scanning all 83 route files finds only `developerTicketRoutes.js` applying bare `authenticate` .
- Payment webhook signatures use `crypto.timingSafeEqual` ; OTP comparison in `beneficiaryController.js:422` does too.
- The scraper device guard fails closed (`scraperRoutes.js:11-13` returns 503 when its key is unset).
- `backend/scripts/security-check.mjs` scans tracked files for secrets; it is currently red (see DATABASE-SECURITY.md DB-001).

---

4. Immediate action plan

#	Action	Finding
1	Gate developer-ticket routes with a real role check	BE-001
2	Make `requireAdminKey` and env-admin fail closed	BE-006
3	Authenticate `/api/customer/*`	BE-007
4	Add `helmet()` and a CSP on the API origin	BE-002
5	Replace wildcard CORS with an allow-list; drop `x-admin-key`	BE-003
6	Rate-limit `/api/auth/login` and `/api/workers/login`	BE-004
7	Authenticate `/uploads` ; serve via an authorising route	BE-005
8	Pin JWT algorithms to `HS256` at all 9 sites	BE-008
9	Remove fallback secrets; bound all token lifetimes	BE-009
10	Route error responses through the generic handler	BE-010
11	Authenticate or remove `/api/fro-count`	BE-011
12	Stream/limit 50 MB uploads	BE-012

5. Verification

```
# Secrets must not appear in tracked files (red = unfixed).
cd backend && npm run security:check

# Confirm the developer-ticket router carries a role gate.
grep -n "router.use" backend/src/routes/developerTicketRoutes.js

# Confirm the API routes use authenticateRole.
grep -rn "authenticateRole" backend/src/routes | wc -l
```